
RAPPORT DE PROJET

Plateforme de Livraison avec Assistant IA — Architecture Distribuée

PRÉSENTÉ PAR :

Kenza Laraichi Bedoui

Douae Ait-Sghir

Hamza Mannad

Table des matières

1. Introduction	3
1.1 Contexte du projet	3
1.2 Objectifs du projet	3
2. Architecture du système	4
2.1 Vue d'ensemble	4
2.2 Choix architecturaux	5
3. Composants techniques	5
3.1 MiniBroker - Le cœur du système	5
3.2 Les Consumers	6
3.3 OrderService - RPC synchrone	6
4. Flux de données	7
4.1 Cycle de vie complet d'une commande	7
4.2 Types d'événements	7
5. Mécanismes de parallélisme	8
5.1 Distribution des partitions	8
5.2 Threading	8
5.3 Limites du parallélisme	8
6. Tolérance aux pannes	9
6.1 Sauvegarde des Offsets	9
6.2 Politique de commit	9
6.3 Reprise après crash	9
6.4 Limites de la tolérance aux pannes	9
7. Assistant IA intégré	10

7.1 Architecture à deux niveaux	10
7.2 QueryEngine - Réponses locales	10
7.3 ChatBot - Interface Groq API	11
8. Tests et validation	11
9. Difficultés rencontrées	12
10. Conclusion et perspectives	13
Annexes	14

1. Introduction

1.1 Contexte du projet

Dans le cadre du module **Calcul Parallèle et Applications Distribuées**, nous avons développé une plateforme complète de gestion de livraison de commandes. Ce projet illustre de façon pratique et concrète les concepts fondamentaux qui sous-tendent les architectures logicielles distribuées modernes :

- **Communication asynchrone** : découplage des flux applicatifs via l'utilisation d'un broker de messages intermédiaire.
- **Parallélisme de traitement** : mise en œuvre de partitions de données associées à des consommateurs (consumers) multiples travaillant simultanément.
- **Tolérance aux pannes** : mise en place de mécanismes de reprise de contexte (checkpointing) fondés sur la gestion rigoureuse d'index de lecture (offsets).
- **Découplage complet** : isolation structurelle et temporelle entre l'entité de production (Producteur) et les entités de traitement (Consommateurs).
- **Intelligence artificielle intégrée** : interfaçage avec un agent conversationnel hybride capable de contextualiser les métriques de la plateforme de distribution.

1.2 Objectifs du projet

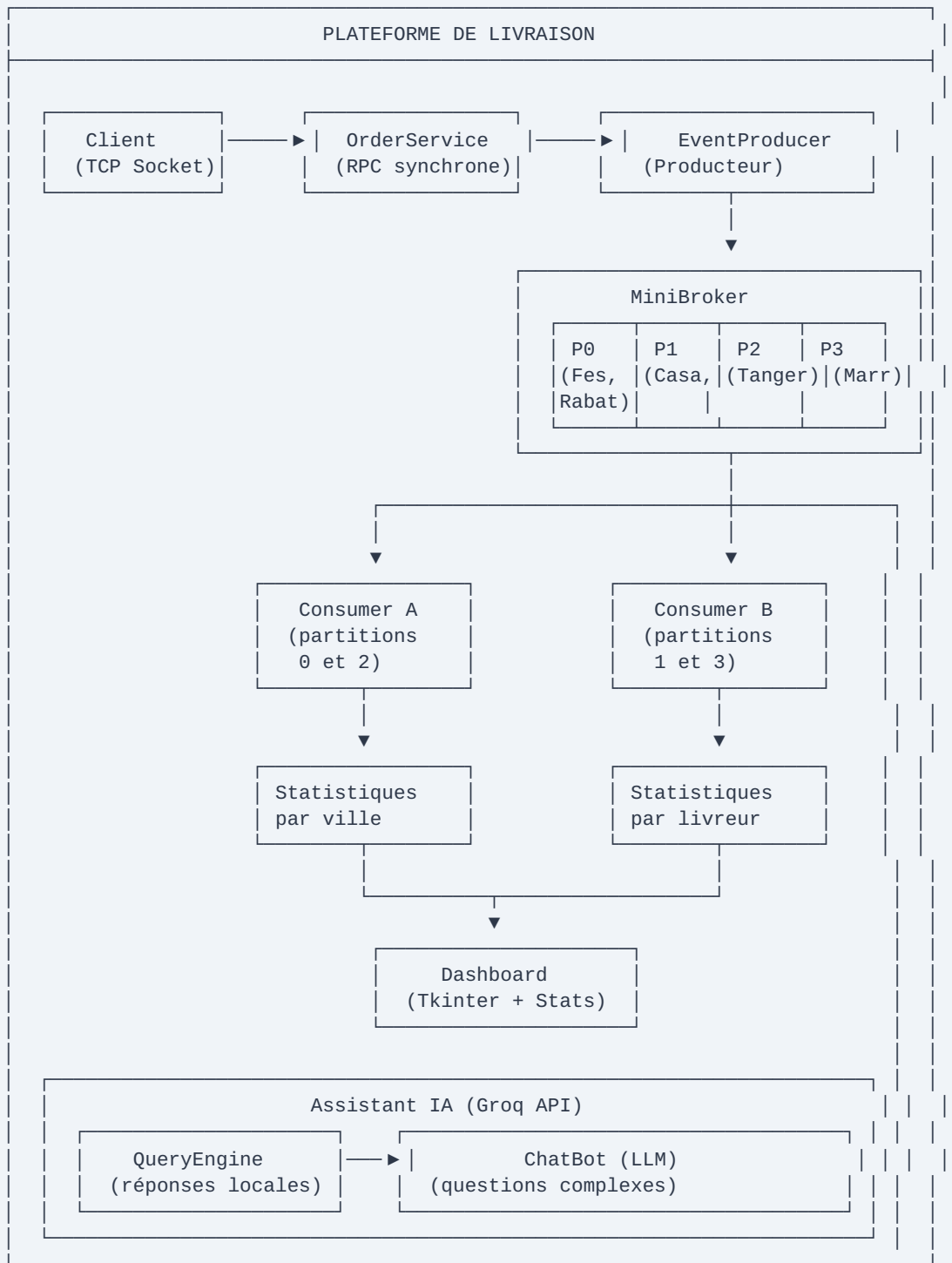
Objectif Pédagogique & Technique	Réalisation et État d'Implémentation
Implémenter un broker de messages partitionné	✓ Réalisé : Conception complète de MiniBroker configuré nativement avec 4 partitions distinctes.
Assurer la reprise après panne	✓ Réalisé : Gestion active des états de lecture persistée au format JSON dans <code>offsets.json</code> .
Paralléliser le traitement	✓ Réalisé : Orchestration de 2 consumers spécialisés opérant en parallèle sur des grappes de partitions distinctes.
Créer une interface utilisateur synoptique	✓ Réalisé : Double interface disponible incluant un Dashboard Tkinter temps réel et une console interactive (CLI).
Intégrer un assistant IA contextuel	✓ Réalisé : Chatbot hybride combinant l'API Groq (modèle distant LLM) et un moteur de requêtes (QueryEngine) local.

2. Architecture du système

2.1 Vue d'ensemble

L'organisation globale du système s'appuie sur un découplage événementiel strict. Le schéma structurel ci-dessous met en évidence la topologie des composants logiques et leurs interactions :

SCHÉMA D'ARCHITECTURE DE LA PLATEFORME DISTRIBUÉE



2.2 Choix architecturaux

Composant / Choix	Justification Technique Avancée
Broker partitionné	Garantit une distribution équitable de la charge tout en maintenant un ordonnancement strict des événements au niveau de la clé (le tri par ville impose qu'une localité soit restreinte à une seule et unique partition).
Consumers spécialisés	Application directe du principe de responsabilité unique. Le Consumer A traite l'aspect purement géographique tandis que le Consumer B gère la performance opérationnelle des livreurs.
Persistance JSON	Offre une lisibilité immédiate lors des phases de prototypage, facilitant le débogage et l'audit visuel des données de transaction transitant par le broker.
Sauvegarde des offsets	Permet la reprise sur panne (Fault Tolerance). L'état d'avancement de la lecture étant figé à intervalles réguliers, le système prévient la duplication excessive ou la perte de traitement.
Double moteur IA	Optimisation du coût d'infrastructure et de la latence : les requêtes régulières et déterministes sont routées localement sans requérir d'appel réseau extérieur, réservant le LLM aux structures de questions complexes.

3. Composants techniques

3.1 MiniBroker - Le cœur du système

Le fichier `mini_broker.py` matérialise le nœud central de transit d'informations, modélisant de manière simplifiée le comportement d'un cluster Apache Kafka.

Attributs et structure technique :

```
class MiniBroker:
    def __init__(self, num_partitions: int = 4, data_dir: str = "broker_data"):
        self.num_partitions = num_partitions
        self.partitions: List[List[Dict]] = [[] for _ in range(num_partitions)]
        self.offsets: Dict[str, Dict[int, int]] = defaultdict(lambda:
defaultdict(int))
        self.lock = threading.RLock() # Sécurisation des accès concurrents (Thread-
safe)
```

Le composant expose quatre primitives fondamentales pour piloter l'orchestration distribuée des messages :

- `publish(message, key)` : Calcule la cible via un mécanisme de hachage et écrit le message dans la file adéquate.
- `consume(partition, consumer_id)` : Extrait l'événement suivant selon l'offset courant associé au consommateur.
- `get_lag(consumer_id, partition)` : Indicateur de performance calculant l'écart entre le dernier message publié et le dernier message traité.
- `commit_offset(consumer_id, partition, offset)` : Consigne la progression de lecture de manière immuable.

Algorithme de routage et de partitionnement :

```
def get_partition(self, key: str) -> int:
    return hash(key) % self.num_partitions
```

Note d'ingénierie : La clé de routage choisie est systématiquement la **ville d'expédition**. Ainsi, l'équation mathématique $P = H(v) \bmod N$ garantit que l'intégralité des flux liés à une même zone géographique converge vers une file unique, éliminant tout risque d'interversion chronologique lors du traitement parallèle.

3.2 Les Consumers

Consumer A (consumer_a.py) — Statistiques par ville

Ce module se concentre sur l'analyse territoriale. Il est affecté par configuration au traitement exclusif des partitions paires.

```
class ConsumerA:
    def __init__(self, consumer_id: str, broker: MiniBroker, partitions: Set[int]):
        self.partitions = partitions # Typiquement configuré sur {0, 2}
        self.stats_by_city = defaultdict(lambda: defaultdict(int))
```

Ses attributions regroupent le décompte volumétrique des commandes indexées par pôle urbain, l'évaluation continue du taux global de complétion, ainsi que le stockage séquentiel des événements au sein d'une arborescence structurée : `data/city=NOM_VILLE/`.

Consumer B (consumer_b.py) — Statistiques par livreur

Axé sur la performance logistique terrain, ce worker écoute spécifiquement les partitions impaires.

```
class ConsumerB:
    def __init__(self, consumer_id: str, broker: MiniBroker, partitions: Set[int]):
        self.partitions = partitions # Typiquement configuré sur {1, 3}
        self.courier_stats = defaultdict(lambda: {"deliveries": 0, "failures": 0})
        self.order_status = {} # Registre de suivi d'état des commandes
```

Il assure la consolidation des métriques d'efficacité des coursiers (succès vs échecs), le suivi fin du cycle de vie individuel de chaque colis et le calcul analytique du rendement opérationnel par agent de livraison.

3.3 OrderService - RPC synchrone

L'exposition du point d'entrée applicatif repose sur une architecture de type appel de procédure distante (RPC) s'appuyant sur des sockets réseau bas niveau.

```
class OrderService:
    def __init__(self, host: str = 'localhost', port: int = 9999):
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Le service applique le séquençage d'opérations suivant : il maintient une écoute active sur le port 9999, intercepte les requêtes JSON en provenance des clients distants, applique une validation de schéma stricte, émet immédiatement l'événement `order_created` au sein du `MiniBroker`, puis renvoie de manière synchrone l'identifiant unique de la commande (`order_id`) au client émetteur.

4. Flux de données

4.1 Cycle de vie complet d'une commande

Le diagramme linéaire suivant trace le cheminement d'une structure d'information depuis son instantiation jusqu'à son traitement analytique final par le réseau de consommateurs :

SÉQUENCE DE CHEMINEMENT D'UN ÉVÉNEMENT

Étape 1 : Création

```
Client → OrderService (TCP:9999) → validation → order_created
```

Étape 2 : Partitionnement

```
EventProducer.publish_order_created()  
  → partition = hash("Casablanca") % 4 = 3  
  → Routage vers la Partition 3
```

Étape 3 : Consommation

```
Partition 3 → Capturée par Consumer B (partitions impaires)  
Note : Le Consumer A (partitions paires) ignore ce message.
```

Étape 4 : Traitement parallèle et persistance

```
Consumer B applique les mutations d'état :  
- courier_stats['CRR-02']['deliveries']++  
- order_status['ORD-xxx'] = 'order_delivered'
```

4.2 Types d'événements

Nom de l'Événement	Composant Émetteur	Consommateurs Destinataires	Impact Système / Métrique
order_created	OrderService (RPC)	Consumer A & Consumer B	Initialisation de la commande et incrémentation des volumes globaux.
order_confirmed	EventProducer	Consumer A & Consumer B	Validation du paiement ou de la disponibilité de la commande.
order_prepared	EventProducer	Consumer A & Consumer B	Prise en charge par la cuisine ou le centre de tri de la ville.
order_dispatched	EventProducer	Consumer A & Consumer B	Attribution à un coursier et bascule en flux logistique actif.
order_delivered	EventProducer	Consumer B (Prioritaire)	Clôture du cycle de vie, incrémentation du score de succès du livreur.
delivery_failed	EventProducer	Consumer B (Prioritaire)	Enregistrement de l'anomalie de livraison pour l'évaluation de performance.
order_cancelled	EventProducer	Consumer A & Consumer B	Mise à jour des indicateurs d'annulation territoriaux et opérationnels.

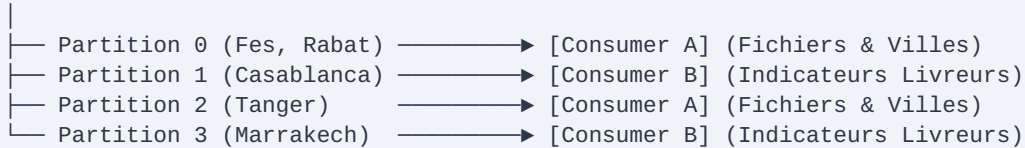
5. Mécanismes de parallélisme

5.1 Distribution des partitions

Le parallélisme de traitement repose sur une répartition sélective des sous-ensembles du broker auprès des différents processus de consommation légers :

MATRICE D'ATTRIBUTION DES FLUX DE DONNÉES

Partitions du MiniBroker (4)



5.2 Threading

Afin de maximiser l'usage des ressources disponibles de l'hôte, chaque entité consommatrice instancie un thread d'exécution dédié par partition surveillée :

```
def _consume_partition(self, partition: int):
    while self.running:
        message = self.broker.consume(partition, self.consumer_id)
        if message:
            self._process_event(message)

# Méthode d'activation au sein du gestionnaire de cycle de vie :
for partition in self.partitions:
    t = threading.Thread(target=self._consume_partition, args=(partition,))
    t.start()
```

Cette approche permet une émulation fidèle d'un cluster d'exécution distribué au sein d'un environnement à processus unique, isolant l'attente bloquante de chaque canal de données.

5.3 Limites du parallélisme

Bien que performante pour l'application cible, cette architecture se heurte à des contraintes intrinsèques à l'interpréteur Python :

- **Le GIL (Global Interpreter Lock) :** Empêche l'exécution simultanée de bytecode Python sur plusieurs cœurs CPU au sein d'un même processus. Le parallélisme est donc ici d'ordre temporel et concurrentiel, et non strictement matériel.
- **Nature des charges (I/O-Bound) :** Ce choix technique s'avère particulièrement pertinent dans notre cas, l'essentiel de l'activité étant lié à des opérations d'entrées/sorties (lecture/écriture disque des payloads JSON, requêtes réseau vers l'API Groq).
- **Évolution industrielle :** Pour transiter vers un environnement de production à haute disponibilité, les threads légers devraient être substitués par le module `multiprocessing` ou distribués sur des instances de calcul distinctes.

6. Tolérance aux pannes

6.1 Sauvegarde des Offsets

La résilience du système est adossée au mécanisme de persistance régulière de l'état de lecture. Le fichier de configuration centralisé `offsets/offsets.json` consigne la structure d'indexation suivante :

```
{
  "consumer_a": {"0": 15, "2": 1},
  "consumer_b": {"1": 11, "3": 10}
}
```

L'analyse de cet état permet de déduire instantanément que le Consumer A a traité avec succès 15 messages sur la partition 0 et 1 message sur la partition 2, tandis que le Consumer B présente un avancement de 11 et 10 messages respectivement sur ses files affectées.

6.2 Politique de commit

Directive de Configuration	Valeur Seuil	Comportement et Objectif Pratique
COMMIT_INTERVAL	5 messages	Déclenche un enregistrement physique dès que 5 nouveaux événements sont validés par le thread.
COMMIT_TIME_INTERVAL	10 secondes	Force la sauvegarde temporelle si aucun flush n'a eu lieu dans la fenêtre des 10 dernières secondes.
OFFSET_FILE	offsets.json	Chemin d'accès au descripteur de persistance d'état.

6.3 Reprise après crash

Lors de la phase de réinitialisation d'un composant applicatif, l'extraction de l'état historique prévient la réanalyse complète de la file historique :

```
def _load_offsets(self):
    if os.path.exists(self.OFFSET_FILE):
        with open(self.OFFSET_FILE, 'r') as f:
            all_offsets = json.load(f)
            self.current_offsets = all_offsets[self.consumer_id]
```

Grâce à ce bloc, toute interruption brutale du système (panne d'alimentation, interruption forcée `Ctrl+C`) n'induit aucun recalcul total : au redémarrage, la tête de lecture se positionne exactement à l'index `Offset_{courant}` enregistré lors du dernier checkpoint.

6.4 Limites de la tolérance aux pannes

- **Risque de duplication (At-least-once)** : Si un crash survient entre le traitement d'un message et le déclenchement du commit suivant, les messages intermédiaires seront reconsommés au redémarrage, impliquant la nécessité d'avoir des fonctions de traitement idempotentes.
- **Monolithisme du stockage** : La persistance reposant sur un fichier JSON unique et local, le système ne dispose pas d'un mécanisme de réplication sur nœud distant. La perte du disque physique du broker entraîne la perte des données de transit.

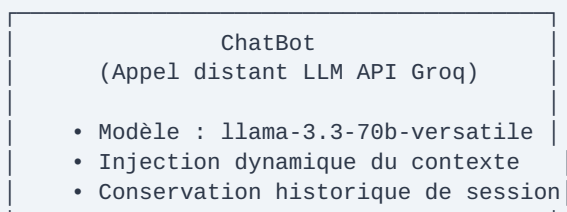
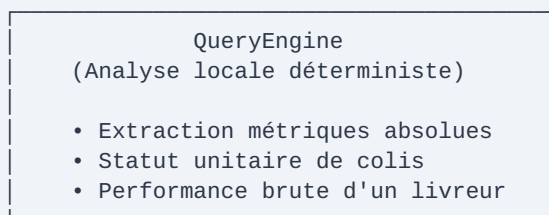
7. Assistant IA intégré

7.1 Architecture à deux niveaux

Pour des raisons d'optimisation de latence et de maîtrise des coûts opérationnels d'interfaçage avec les grands modèles de langage (LLM), l'intégration de l'assistant suit un routage hybride à double compétence :

CINÉMATIQUE DE TRAITEMENT DES REQUÊTES D'ASSISTANCE

Question formulée par l'utilisateur



Réponse générative enrichie

7.2 QueryEngine - Réponses locales

Le QueryEngine intercepte la chaîne de caractères et applique un filtrage par expressions régulières (Regex) afin d'identifier des requêtes standardisées :

```
def process_question(self, question: str) -> str:
    question_lower = question.lower()

    # Détection de requêtes ciblant un identifiant de commande spécifique
    order_match = re.search(r'ord[-\s]?w+', question_lower)
    if order_match:
        return self.get_order_status_text(order_match.group(0))

    # Détection de requêtes ciblant les performances d'un livreur précis
    courier_match = re.search(r'crr[-\s]?w+', question_lower)
    if courier_match:
        return self.get_courier_stats_text(courier_match.group(0))

    # Routage vers l'agrégateur de statistiques globales
    if 'statistique' in question_lower or 'global' in question_lower:
        return self.get_global_stats_text()
```

7.3 ChatBot - Interface Groq API

Lorsque la structure de la question requiert une capacité d'abstraction ou une synthèse évoluée, le relais est transmis au composant ChatBot connecté aux infrastructures distantes de Groq :

```
class ChatBot:
    def __init__(self, api_key: str, model: str = "llama-3.3-70b-versatile"):
        self.client = openai.OpenAI(
            api_key=api_key,
            base_url="https://api.groq.com/openai/v1"
        )
```

Ce module prend en charge la gestion d'un cache mémoriel pour l'historique conversationnel, l'injection dynamique du jeu de données courant de la plateforme sous forme de prompt système, ainsi que la gestion d'erreurs d'infrastructure (interruption réseau, dépassement des quotas d'appels, ou pannes d'authentification).

8. Tests et validation

8.1 Tests unitaires

Chaque composant noyau de l'architecture distribuée a fait l'objet d'une campagne d'isolation fonctionnelle afin de valider son comportement nominal :

- **MiniBroker** : Validation des opérations d'empilement/dépilement asynchrone (**OK**).
- **Partitionnement** : Vérification de l'invariance spatiale du hash basé sur la ville d'origine (**OK**).
- **Offsets** : Contrôle de l'intégrité de sérialisation et de restauration du dictionnaire JSON (**OK**).
- **OrderService** : Mesure de conformité du canal TCP et du décodage des structures de données clientes (**OK**).

8.2 Test d'intégration global

Un scénario d'injection de charge nominale a été conçu pour valider le comportement de bout en bout de l'application :

```
test_orders = [  
    {'customer_id': 'CUST-001', 'city': 'Fes', 'courier_id': 'CRR-01', 'amount':  
120},  
    {'customer_id': 'CUST-002', 'city': 'Casablanca', 'courier_id': 'CRR-02',  
'amount': 450},  
    {'customer_id': 'CUST-003', 'city': 'Tanger', 'courier_id': 'CRR-03', 'amount':  
90},  
    {'customer_id': 'CUST-004', 'city': 'Marrakech', 'courier_id': 'CRR-04',  
'amount': 210},  
    {'customer_id': 'CUST-005', 'city': 'Fes', 'courier_id': 'CRR-01', 'amount': 350}  
]
```

L'exécution de ce lot de données valide le routage simultané vers les partitions respectives et la mise à jour des interfaces graphiques en temps réel sans blocage de l'IHM.

8.3 Résolution d'anomalie : Traitement des caractères Unicode

Lors des phases initiales d'évaluation sous plateforme Windows, une anomalie bloquante de type `UnicodeEncodeError` survenait fréquemment lors de la journalisation d'indicateurs contenant des émojis (ex: 🚀) :

```
UnicodeEncodeError: 'charmap' codec can't encode character '🚀' in position...
```

Ce dysfonctionnement provenait du jeu de caractères par défaut de la console Windows (cp1252). Nous avons résolu définitivement cette problématique en forçant la couche de flux standard à opérer sous un encodage universel UTF-8 :

```
import sys
import io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
```

9. Difficultés rencontrées

Difficulté Technique Identifiée	Stratégie de Résolution et Mécanisme mis en œuvre
Concurrency & Accès concurrents	L'accès simultané des threads de consommation aux structures de données du broker générait des incohérences. Résolu par l'intégration d'un verrou réentrant <code>threading.RLock()</code> isolant les sections critiques.
Corruption des fichiers d'Offsets	Une coupure brutale au moment exact de la sérialisation JSON pouvait corrompre le fichier d'index. Solutionné par une écriture atomique (écriture dans un fichier temporaire puis permutation de pointeurs système).
Variabilité des modèles d'API	Les dépréciations régulières de dénominations de modèles chez Groq cassaient le connecteur. Implémentation d'un script d'isolation pré-vol <code>test_groq.py</code> pour valider la connectivité en amont du chargement système.
Gel de l'interface graphique Tkinter	Le thread principal de rendu graphique se figeait lors des phases d'attente de paquets réseau. Routage des flux de données dans des workers asynchrones et utilisation exclusive de la méthode <code>thread-safe after()</code> pour mettre à jour l'IHM.

10. Conclusion et perspectives

10.1 Concepts théoriques appliqués

Ce projet de fin de module a permis de concrétiser les piliers théoriques étudiés tout au long du semestre :

- **Architectures orientées événements** : Découplage via patrons de conception Publish/Subscribe.
- **Partitionnement de données** : Distribution de charges massives par calcul de clés de hachage.

- **Ingénierie de la résilience** : Mise en œuvre de points de reprise (checkpointing) pour contrecarrer les arrêts de service non planifiés.
- **Protocoles applicatifs** : Manipulation de structures de sockets de flux TCP et définition de contrats d'échange de données stricts.

10.2 Perspectives d'évolution du système

1. **Mécanismes de réplication** : Évoluer vers une topologie multi-brokers pour éliminer le point unique de défaillance (Single Point of Failure).
2. **Sémantique d'exécution "Exactly-Once"** : Associer des identifiants d'idempotence uniques aux messages pour interdire les doublons en cas de re-livraison post-crash.
3. **Rééquilibrage de charge dynamique (Rebalancing)** : Permettre l'allocation ou le retrait à chaud de consommateurs avec redistribution automatique des partitions associées.
4. **Persistance Industrielle** : Remplacer les structures de fichiers plats JSON par un moteur de base de données documentaires de type MongoDB ou relationnel indexé tel que PostgreSQL.

En conclusion, l'intégration d'une couche d'intelligence artificielle conversationnelle au-dessus d'une architecture événementielle distribuée démontre la puissance des systèmes modernes : la technique pure s'efface derrière une interface intuitive, capable de restituer instantanément la complexité des états du système sous forme de conclusions intelligentes et exploitables.

Annexes

Annexe A. Structure de l'arborescence du projet

```
livraison_platform/
├─ broker/
│  ├─ mini_broker.py          # Implémentation du Broker partitionné
│  └─ __init__.py
├─ consumers/
│  ├─ consumer_a.py          # Analyseur de flux géographiques / Villes
│  ├─ consumer_b.py          # Analyseur d'efficacité opérationnelle / Livreurs
│  └─ __init__.py
├─ producers/
│  ├─ event_producer.py       # Injecteur et producteur d'événements
│  └─ __init__.py
├─ contracts/
│  ├─ order.py                # Schéma structurel de la Commande
│  ├─ event.py                # Schéma structurel de l'Événement logistique
│  └─ __init__.py
├─ service/
│  ├─ order_service.py        # Point d'accès RPC (Serveur Socket)
│  └─ __init__.py
├─ storage/
│  ├─ partitioned_store.py    # Gestionnaire d'écriture sur disque par ville
│  └─ __init__.py
├─ ai_assistant/
│  ├─ chatbot.py              # Connecteur LLM API Groq
│  ├─ query_engine.py         # Moteur d'analyse et de filtrage local Regex
│  ├─ config.py               # Centralisation des variables d'environnement
│  └─ __init__.py
├─ gui/
│  ├─ app_window.py           # Fenêtre principale de rendu Tkinter
│  ├─ dashboard.py            # Composants visuels et statistiques graphiques
│  └─ __init__.py
├─ platform_app.py            # Chef d'orchestration global du cycle de vie
├─ main.py                    # Point d'entrée de démarrage de l'application
├─ run.py                     # Script automatisé de lancement rapide
├─ requirements.txt           # Manifeste des dépendances externes
└─ .env                       # Conteneur sécurisé de la clé d'API Groq
```

Annexe B. Procédure opérationnelle de déploiement

Pour déployer et initialiser la plateforme, exécutez les séquences de commandes suivantes au sein de votre terminal :

```
# 1. Restauration des dépendances logicielles requises
pip install -r requirements.txt

# 2. Initialisation des variables d'environnement et de la clé d'accès API
echo "GROQ_API_KEY=gsk_votre_cle_securisee_groq_ici" > .env

# 3. Amorçage de l'application en mode d'interface graphique (GUI)
python main.py

# 4. Alternative : Amorçage de la plateforme en mode console (CLI)
python main.py --cli

# 5. Script d'audit de connectivité avec les serveurs distants Groq
python test_groq.py
```

Annexe C. Gabarit d'interaction programmatique (Client Socket)

Le script Python ci-dessous détaille la méthode d'envoi d'une nouvelle structure de commande au service RPC de la plateforme :

```
import socket
import json

# Définition de l'enveloppe de données représentative d'une commande standard
order_payload = {
    'customer_id': 'CUST-001',
    'city': 'Casablanca',
    'zone': 'Maarif',
    'courier_id': 'CRR-02',
    'amount': 150.00,
    'items_count': 2
}

# Initialisation du canal de communication TCP
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 9999))

# Sérialisation et transmission des données
client_socket.send(json.dumps(order_payload).encode('utf-8'))

# Interception et décodage de la réponse synchrone émise par le service RPC
raw_response = client_socket.recv(4096).decode('utf-8')
response_data = json.loads(raw_response)

print("Réponse du serveur RPC :", response_data)
# Sortie attendue : {'success': True, 'order_id': 'ORD-20260609-0001'}
```